

Past, Current and Future Trend in System Software Methodology

Afiq Danial Bin Rozaidie, Pravinraj a/l Sivabathi, Yoke Yu Lam

Faculty of Computing, University of Technology Malaysia

¹danial-04@graduate.utm.my

²pravinraj04@graduate.utm.my

³lam.yu@graduate.utm.my

Abstract— The field of software development has evolved significantly from the traditional Monolithic Architecture and Waterfall Methodology to the current Agile and User Centred Design. This paper explores the trend of the past and current trends in system software methodology and examines future trends, including the integration of Artificial Intelligence in software development processes and increasing emphasis on security.

Keywords— Software Development, SDLC, Monolithic Architecture, Waterfall Model, Agile Methodology, User-Centred Design, Artificial Intelligence, Security, DevSecOps

I. INTRODUCTION

Software Development is a complex field that encompasses the process of designing, creating, testing, and maintaining software applications and systems.

The journey of software development over the past two decades has been nothing short of revolutionary. From the early days of monolithic architectures or waterfall models to today's microservices and cloud-native applications or quantum computing in the distant future, the landscape has continually evolved.

As we look towards the future, it is crucial to understand where we have been, where we are, and where we are headed as the technologies keep evolving and become more advanced.

II. PAST TRENDS IN SYSTEM SOFTWARE

A. The Era of Monolithic Architecture

Like in the early 2000s indeed marked the era of monolithic applications in the realm of software development and was the standard for deploying applications. Monolithic architecture refers to building an application as a single unit, typically containing the user interface, business logic, and data access layers all bundled together [1].

As shown in Figure 1, this architecture components are connected, frequently using the same database and codebase. But because growing a monolith requires reproducing the entire application, which can be laborious and resource-intensive, tight coupling can pose scalability issues. Because of its intrinsic structure, monolithic architecture may have trouble expanding and meeting growing demands, even though it is easy to design and maintain [2].

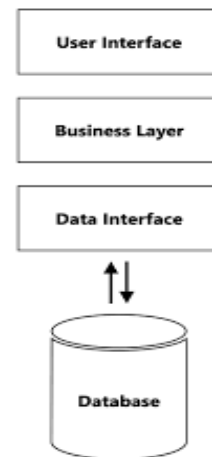


Figure 1 Monolithic Architecture [2]

During that time, monolithic applications were a dominant approach by a several factors which are:

1) *Simplicity*: Monolithic applications were relatively easier to develop, deploy, and manage compared to distributed systems [1].

2) *Technology Constraint*: The technology landscape during that time was not as advanced as it is today. Microservices architecture, which promotes breaking down an application into smaller, independent services, was not as popular or well-understood back then [1].

Despite providing the best approach, the software systems tend to grow larger and more complex. Therefore, monolithic architectures start to show the limitations, which is that a single large codebase can be difficult to maintain and understand [3]. With all components tightly coupled, making changes to one part of the application can have unintended consequences elsewhere. This makes maintenance and updates more difficult over time.

B. The Waterfall Development Model

The Waterfall development model was introduced in 1970 by Winston W. Royce, a software engineer at Lockheed, in his paper titled "*Managing the Development of Large Software Systems*." Although Royce presented the model to describe the

linear nature of software development, he acknowledged its limitations and suggested iterative improvements. Ironically, despite his critique, the Waterfall model was widely adopted in the following decades because it was simple to understand and aligned with traditional engineering practices used in construction and manufacturing.

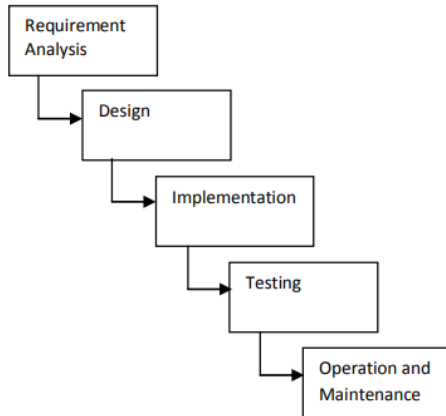


Figure 2 The phases of a Waterfall Model [4]

In the early version, waterfall style breaks up projects based on activities which are requirement analysis, design, coding, and testing. While the other version consists of activities: communication, planning, modelling, construction and deployment [4]. Summarily, the waterfall could be said to be involved with the following phases: requirement analysis, design, implementation, testing, and operation and maintenance such as Figure 2 above.

The Waterfall development model was a groundbreaking framework in the early days of software engineering, offering a structured and predictable approach for building complex systems. While it has been largely replaced by Agile in many industries, Waterfall is still relevant in highly regulated and safety-critical environments.

In summary, Waterfall laid the foundation for systematic software development practices but eventually gave way to more adaptive approaches to meet the demands of modern software.

III. PRESENT TRENDS IN SYSTEM SOFTWARE METHODOLOGY

A. Agile Methodology

Agile is an iterative and incremental approach to software development that emphasises collaboration, flexibility, and continuous delivery of functional software. Agile projects are broken down into several dynamic phases, manageable tasks, or "sprints." Sprints is a dedicated period in which a set amount of work will be completed on a project. The purpose of this model is to deliver functional software products in small increments while making changes as necessary [6].

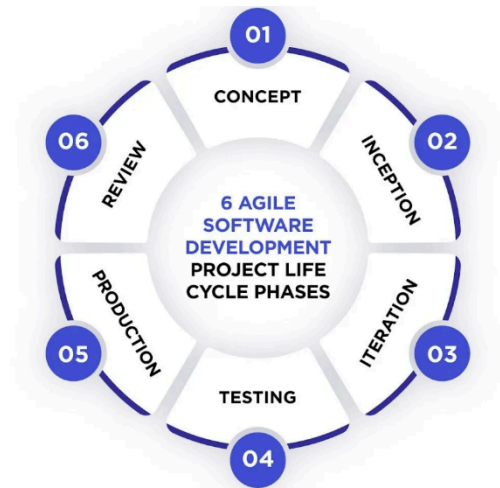


Figure 3: The Phases of Agile Model [7]

1) *Concept*: Phase 1 is about defining the project's objectives. Stakeholders and users come together to define the project's goals and identify the user target. This phase focused on timelines, assessing costs and key features to ensure the project's feasibility [8].

2) *Inception*: After clarifying the scope, it became important to assemble a team. Resources were gathered next. Each member received specific roles, tools and technologies are identified. The team worked together to ensure everyone understood their tasks clearly. This phase involves creating a detailed plan for managing and executing the project [8].

3) *Iteration*: This phase lasted the longest in the Agile process. The team worked in sprints. Each sprint focused on a specific set of tasks from the product backlog to deliver a working piece of the product with each cycle. Iteration promotes adaptability and continuous improvement. The team would ask users for their feedback and make changes accordingly [8].

4) *Testing*: After each sprint, thorough testing was done to ensure everything works correctly. This includes unit tests, integration tests, acceptance tests, and system tests. The unit testing was mainly the performance of the front and back ends. Integration testing tested the different components for compatibility. Acceptance testing ensures that requirements of the user were met and then system testing to check the overall functionality. User feedback shaped design decisions, and effective communication ensured the final product met expectations, delivering a smooth and reliable user experience [8].

5) *Release*: Once testing and refinement thoroughly, the product becomes ready for release. User training helped customers get started with the new software. After all activities were refined, the product was released to the public. Maintenance ensures smooth operation afterward [8].

6) *Review*: The final phase focuses on continuous improvement, driven by user feedback and performance monitoring. The team analyses user experiences, identifies areas for enhancement, and updates the product accordingly. The development process itself is also continuously refined based on user feedback throughout the project lifecycle [8].

B. User Centred Design (UCD)

UCD is an iterative process where designers focus on the users and their needs. Design teams engage with the users to gain understanding of user behaviours, needs, and pain points [9]. The methodology goes through four phases and repeats in a cyclic manner until such a point that the design fully meets users' needs [9].

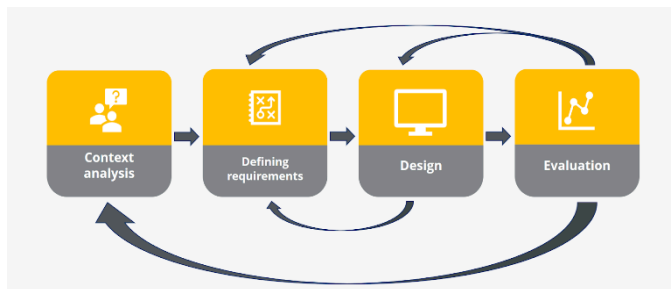


Figure 4 The UCD Model [9]

1) *Context Analysis*: Designers draw on how and where users will interact with the system. It includes an understanding of the user's environment, goals, tasks, and any constraints they might face [9].

2) *Defining Requirements*: The designer will define the user requirements based on the context, specifying what users expect from the system and consolidating key features and functionalities that meet those needs [9].

3) *Solution Design*: The design team develops prototypes, wireframes, or mock-ups to present possible solutions. These designs should solve the problems identified during the previous phase and meet the users' requirements [9].

4) *Design Testing*: Assessment such as usability testing and feedback sessions are conducted to test the suggested solutions.

5) *Evaluation*: The team evaluates how the design works for the users. They cross-check how well the design performs in comparison to the needs and context of the users. If the design isn't quite right, changes are made, and the process is repeated by the team until the design satisfies the needs of the users to the fullest.

IV. FUTURE TRENDS IN SOFTWARE DEVELOPMENT METHODOLOGY

A. AI Integration in Software Development

Artificial Intelligence (AI) is expected to become deeply integrated into software development. Today, many tools and techniques have been utilised by developers as an additive integration of AI into SDLC to enhance efficiency. Future methodologies will focus on AI-native SDLCs, where AI seamlessly integrates into every phase [10].

1) AI-native SDLC

- **Knowledge base**: AI can assist with domain-specific tasks in planning, analysis, and design phase by capturing knowledge from various sources and identifying knowledge gaps [10]. AI would act as a repository of expertise.
- **Faster Development**: With the aid of AI such as AI coding assistant and Large Language Models, can significantly reduce development time and automating tasks. The integration of AI simplifies software development, maintenance, bug detection, and documentation [10].
- **Shift in Developer roles**: As the AI models continue to evolve and improve, human developers may transition from creation to verification and refinement [10].

2) *V-Bounce Model*: To leverage the potential of AI in software development, [10] proposed a V-Bounce model adapted from the V-model. While the V-model incorporates testing throughout the development process, the V-Bounce model reduces the coding time with the integration of AI. The focus of the V-Bounce model is given to the requirements, design, and testing phase.

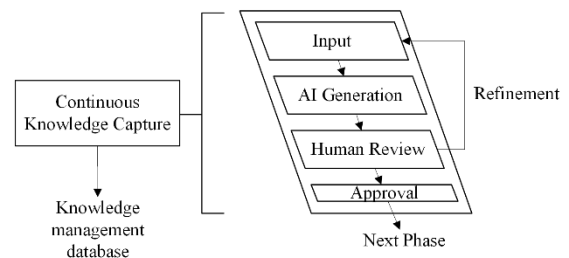


Figure 5 The Anatomy of a V-Bounce Phase [10]

- **Continuous testing**: AI would generate tests as requirements are being created. This allows early detection of ambiguities or inconsistencies in the requirements [10].
- **Iterative Refinement**: In each phase, the output from the AI model would be verified and refined before the next phase [10].
- **Knowledge Capture**: Throughout the process, AI captures new insights, which can be applied in the future [10].

B. Increased Focus on Security in System Development

Despite advancements, system software remains vulnerable to cyberattacks. Traditional software development methodologies often treat security as an afterthought, causing inappropriate security controls to be implemented [11].

1) *SecSDM*: [12] proposed a secure software development methodology, *SecSDM*, which embeds security aspects throughout the SDLC.

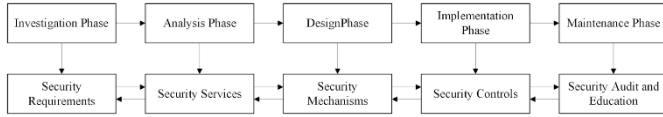


Figure 6 Security in Software Development Life Cycle [12]

- Investigation: Conduct risk analysis by identifying assets, threats and vulnerabilities [12].
- Analysis: Identify the relevant security services and level of protection required to mitigate risk [12].
- Design: Map and consolidate security services to security mechanisms [12].
- Implementation: Map security mechanisms to software security components [12].
- Maintenance: Ensure the software is used in a secure manner [12].

2) *DevSecOps*: Continuous Integration has emerged to eliminate discontinuities between development and deployment such as DevOps and BizDev [13]. To address the security gap in DevOps, *DevSecOps* can be implemented by embedding security protocols directly into the development and deployment pipeline. This introduces security earlier in the SDLC, so that vulnerabilities in the product and the cost to fix them are reduced. [14]

- Continuous Security Integration: The security practices can be done in parallel to the development process and security issues can be fixed sooner in the life cycle [14].
- Automated Security Checks: Integrating security tests into the CI/CD pipeline for automation so that the agility of DevOps is not sacrificed [14].

V. CONCLUSION

Various methodologies have been developed to improve the efficiency of software development and the quality of the final product. From the shift from the traditional Waterfall approach to the current Agile methodology, it is important to acknowledge the rapid advancements in the field. Furthermore, as we consider the unforeseen future, such as the rise of AI and machine learning in software development and the integration of security practices, it is crucial for all practitioners to engage in continuous learning to adapt to the ever-changing technological landscape.

REFERENCES

- [1] Jha, R. (2024, May 26). *Technology Trends and Software Development: past, present, and future – the foundations of modern software development*. <https://www.linkedin.com/pulse/technology-trends-software-development-past-present-future-rajoo-jha-cwwlc/>
- [2] Voloshina, I. (2020). Applying microservices pattern to monolithic software. https://www.theseus.fi/bitstream/handle/10024/858838/Shrestha_Deeps_ikha.pdf?sequence=2
- [3] Khan, S. (2024, April 29). The evolution of software architecture: from monolithic to microservices. *Medium*. <https://medium.com/@ksaquist/the-evolution-of-software-architecture-from-monolithic-to-microservices-6a0ff15ce326#:~:text=The%20Era%20of%20Monolithic%20Architecture&text=All%20components%20of%20an%20application,but%20came%20with%20significant%20limitations.>
- [4] Adenowo, A. A., & Adenowo, B. A. (2013). Software engineering methodologies: a review of the waterfall model and object-oriented approach. *International Journal of Scientific & Engineering Research*, 4(7), 427-434. https://www.researchgate.net/profile/Adetokunbo_Adenowo/publication/n344194737_Software_Engineering_Methodologies_A_Review_of_the_Waterfall_Model_and_Object-Oriented_Approach/links/5f5a803292851c07895d2ce8/Software-Engineering-Methodologies-A-Review-of-the-Waterfall-Model-and-Object-Oriented-Approach.pdf
- [6] Laoyan, S. (2024, February 2). *What is agile methodology? (a beginner's guide)* [2024]. *Asana*. <https://asana.com/resources/agile-methodology>
- [7] A. Dziuba and A. Dziuba, "Navigating the agile software development life cycle: phases, tools, roadmap," *Relevant Software*, Oct. 03, 2024. <https://relevant.software/blog/agile-software-development-lifecycle-phases-explained/>
- [8] Nehra, M. (2022, May 11). *6 Stages of the Agile Development Lifecycle*. [www.decipherzone.com. https://www.decipherzone.com/blog-detail/agile-development-lifecycle](https://www.decipherzone.com/blog-detail/agile-development-lifecycle)
- [9] *What is User Centered Design (UCD)?* (2016, June 5). The Interaction Design Foundation; Interaction Design Foundation. <https://www.interaction-design.org/literature/topics/user-centered-design?srsltid=AfmBOopERjDbIenP5c58Epg3t0RK6VTb5SoV7z8Lp1vJHn3lanF79yv>
- [10] C. Hymel, "The AI-Native Software Development Lifecycle: A Theoretical and Practical New Methodology," 2024. doi: 10.48550/arXiv.2408.03416.
- [11] L. Futcher and R. Von Solms, "SecSDM: A usable tool to support IT undergraduate students in secure software development.," *HAISA*, pp. 86–96, Jun. 2012, [Online]. Available: <https://dblp.uni-trier.de/db/conf/haisa/haisa2012.html#FutcherS12>
- [12] L. Futcher and R. Von Solms, "SecSDM: A Model for Integrating Security into the Software Development Life Cycle," in *IFIP advances in information and communication technology*, 2007, pp. 41–48. doi: 10.1007/978-0-387-73269-5_6.
- [13] S. Alsaqqa, S. Sawalha, and H. Abdel-Nabi, "Agile Software Development: Methodologies and Trends," *International Journal of Interactive Mobile Technologies (iJIM)*, vol. 14, no. 11, p. 246, Jul. 2020, doi: 10.3991/ijim.v14i11.13269.
- [14] D. Anjaria and M. Kulkarni, "Effective DevSECOPS Implementation: A Systematic Literature review," *CARDIOMETRY*, no. 24, pp. 410–417, Nov. 2022, doi: 10.18137/cardiometry.2022.24.410417